UGV Part 1: Progress Log

Yashraj Kranthi
EE26BTECH11235

1 November 2026

Contents

1 Project Overview **2**
1.1 Objective . . . 2
1.2 Hardware Architecture . . . 2
1.3 Working Principle . . . 2
1.4 Design Notes . . . 2
1.5 Planned Features and Scope of Part 1 . . . 3
1.6 Expected Outcome . . . 3
1.7 Bill of Materials . . . 3

2 Progress Log **3**

3 Current Status **14**

1 Project Overview

1.1 Objective

The objective of this project is to design and implement a remotely controlled Unmanned Ground Vehicle (UGV) using an ESP32 microcontroller. The vehicle uses a two-wheel differential-drive mechanism with two DC motors.

The UGV is controlled wirelessly over Bluetooth from an Android smartphone running the Dabble gamepad app. The ESP32 processes the received commands and generates PWM signals that control the speed and direction of the motors through an L293D motor driver.

Part 1 focuses on reliable motor control, smooth acceleration and deceleration, and wireless communication with a safe failure mode.

1.2 Hardware Architecture

- **ESP32 development board:** main controller; receives Bluetooth commands, interprets them and generates PWM and direction signals.
- **L293D motor driver:** dual H-bridge that lets the ESP32 control the direction and speed of the two DC motors.
- **DC motors (2):** provide the mechanical movement for the differential drive.
- **Battery supply:** two 18650 lithium-ion cells in series (nominal 7.4 V, 8.4 V fully charged).
- **Voltage regulator (7805, intended):** provides a regulated 5 V supply for the ESP32 and the L293D logic supply. (See Section 2 for the problem encountered with this part.)
- **Bluetooth communication:** the ESP32's built-in Bluetooth links to the Dabble gamepad on the smartphone.

1.3 Working Principle

1. The user presses a button on the Dabble gamepad on an Android smartphone.
2. The command is transmitted to the ESP32 over Bluetooth.
3. The ESP32 interprets the command and determines the required movement.
4. The ESP32 outputs direction-control signals and PWM signals to the L293D.
5. The L293D drives the two DC motors accordingly.
6. The difference in direction and speed between the two motors determines how the UGV moves (straight, turning, or pivoting).

1.4 Design Notes

- **Logic levels:** the ESP32 GPIO outputs 3.3 V, which is above the L293D's logic-high input threshold, so the ESP32 can drive the L293D inputs directly.
- **Driver voltage drop:** the L293D has a voltage drop of roughly 1.4 V per output stage, so the motors receive less than the battery voltage.

- **Common ground:** the battery negative, ESP32 GND and the L293D ground pins are tied to a single ground rail to give all signals the same reference.
- **Regulator dissipation:** a linear regulator dissipates $P \approx (V_{in} - V_{out})\, I$ as heat, i.e. about $3.4\, I$ watts at 8.4 V in and 5 V out, so it can get hot under load.

1.5 Planned Features and Scope of Part 1

1. Assembly and testing of the motor-driving circuit.
2. Independent PWM speed control of the left and right motors.
3. Forward, reverse, left, right and stop commands.
4. Smooth acceleration and deceleration.
5. Speed adjustment (initially planned as three fixed levels; revised, see Entry 2).
6. Bluetooth communication with a communication-loss stopping mechanism.
7. Basic motor calibration and electrical reliability testing.

1.6 Expected Outcome

A functional two-wheel differential-drive UGV that can be operated remotely over Bluetooth, with independent motor control, adjustable speed, smooth acceleration and deceleration, and a safe stop when the Bluetooth connection is lost.

1.7 Bill of Materials

Table 1: Components used

Component	**Details**	**Qty**
ESP32 development board	Main controller	1
L293D	Dual H-bridge motor driver	1
DC motors with wheels	Differential drive	2
18650 Li-ion cells	Series pack, 7.4 V nominal	2
Voltage regulator	7805 intended (see Section 2)	1
USB cable	Programming the ESP32	1

2 Progress Log

Entry 1: Collecting components

Collected the components from the core labs (Figure 1). I forgot to collect the USB cable even though I had filled in the request form; this delayed uploading code to the ESP32 until Entry 2.

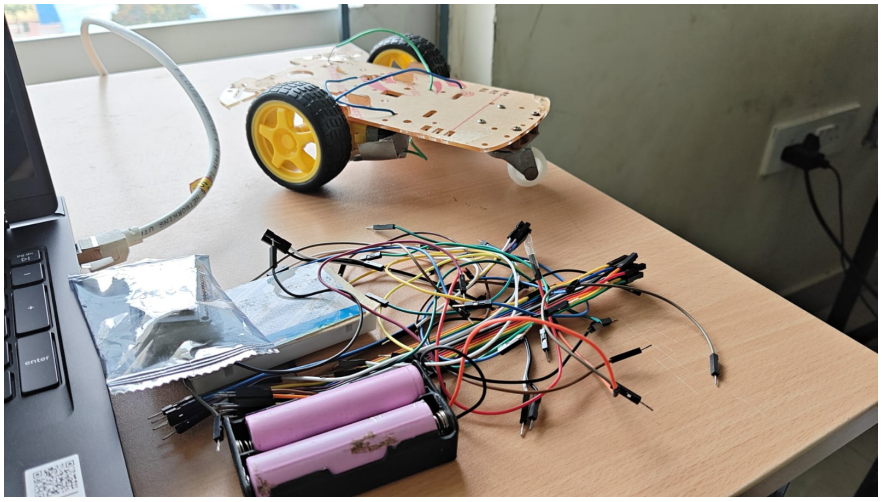


Figure 1: Components collected from the core labs

Entry 2: Wiring the ESP32 and L293D

Primary connections are shown in Figure 2.

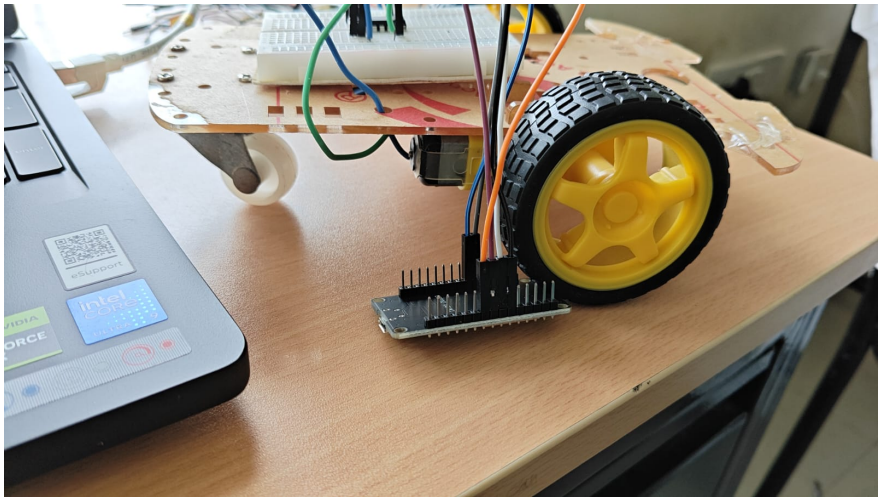


Figure 2: Primary connections to the ESP32

Mistake and correction. I assumed that L293D pins 9–16 were numbered left to right. They are actually numbered right to left: the pins run counter-clockwise starting from the notch. Figure 3 shows the wrong wiring and Figure 4 the corrected version.

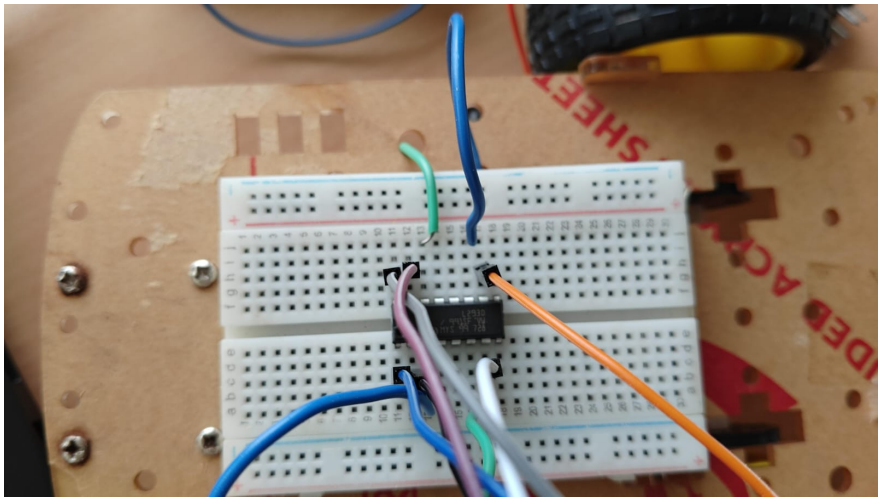


Figure 3: Incorrect wiring caused by a wrong L293D pinout assumption

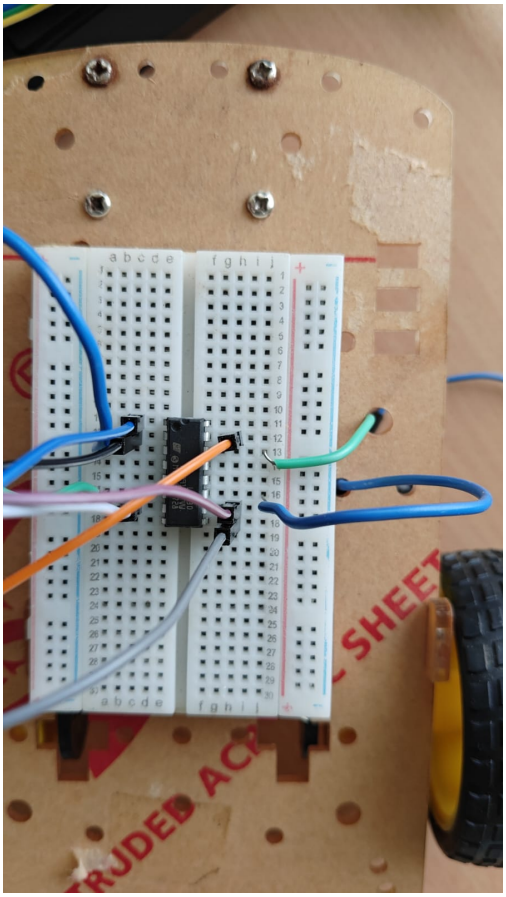


Figure 4: Correctly wired ESP32 and L293D

Table 2: Pin mapping

ESP32 pin	L293D pin	Function
18	1	Left motor PWM / enable (1,2EN)
32	2	Left motor direction input 1 (1A)
33	7	Left motor direction input 2 (2A)
19	9	Right motor PWM / enable (3,4EN)
25	10	Right motor direction input 1 (3A)
26	15	Right motor direction input 2 (4A)

Common ground rail. I connected the negative terminal of the battery, the ESP32 GND and L293D pins 4, 5, 12 and 13 to a common ground rail (Figure 5). The finished wiring is shown in Figure 6.

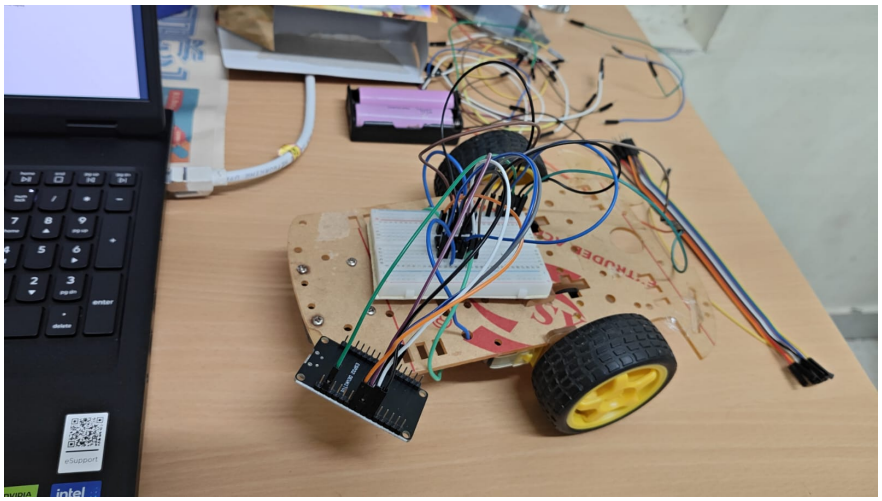


Figure 5: Making the common ground rail

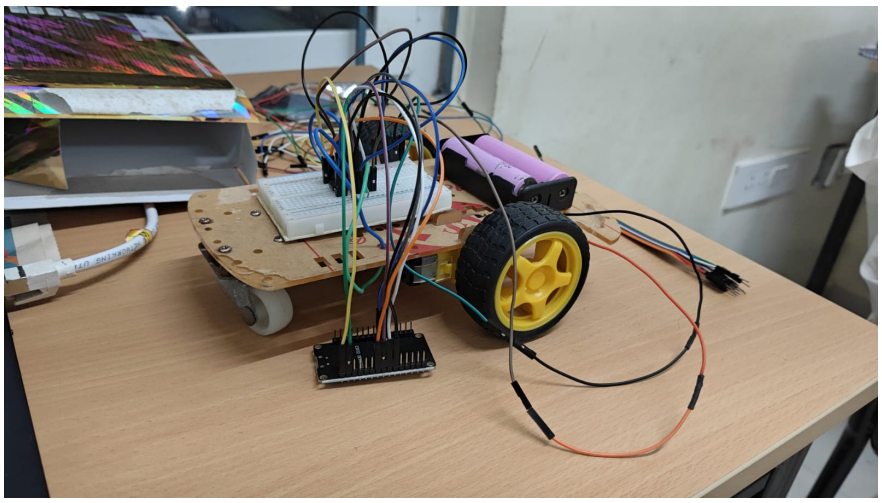


Figure 6: Finished circuit wiring

Entry 3: Checking the two motors with PWM

I checked the rotation direction and speed of each motor under different PWM values using the code in Figure 7.

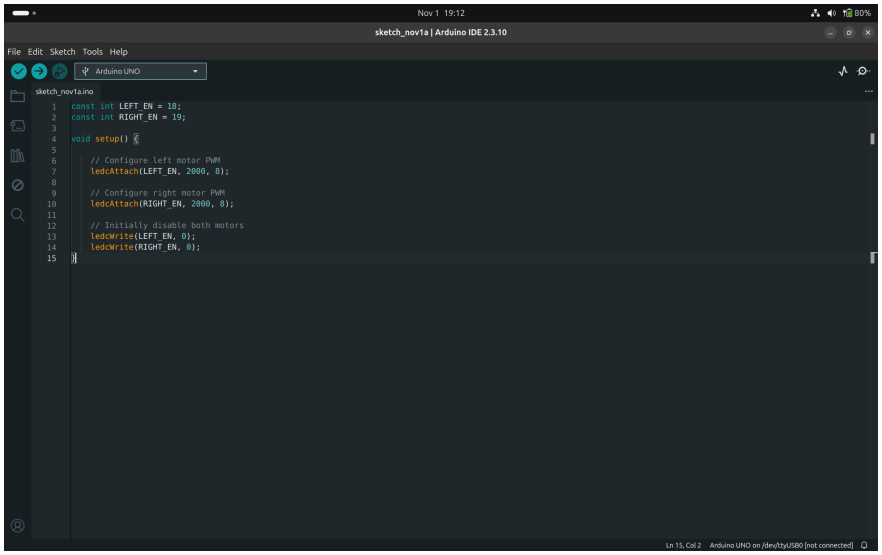


Figure 7: Code used to check individual motor behaviour at different PWM values

This gives us:

1. **Controlled turning:** rotating one wheel faster than the other.
2. **Differential steering:** rotating the wheels in opposite directions to pivot about the centre.
3. **Individual motor control:** starting, stopping and reversing either motor independently.

Table 3: Goal of independent motor control

System	Behaviour
Old	Both motors receive the same fixed power level.
Improved	The ESP32 independently controls the speed and direction of each motor.

This lets the UGV move straight, turn precisely, rotate in place and compensate for differences between the motors.

Entry 4: Prerequisites

USB cable. I obtained the missing USB cable from the core labs (Figure 8).



Figure 8: USB cable obtained from the core labs

Updated wiring. The circuit was updated to power the ESP32 through the regulator (Figure 9).

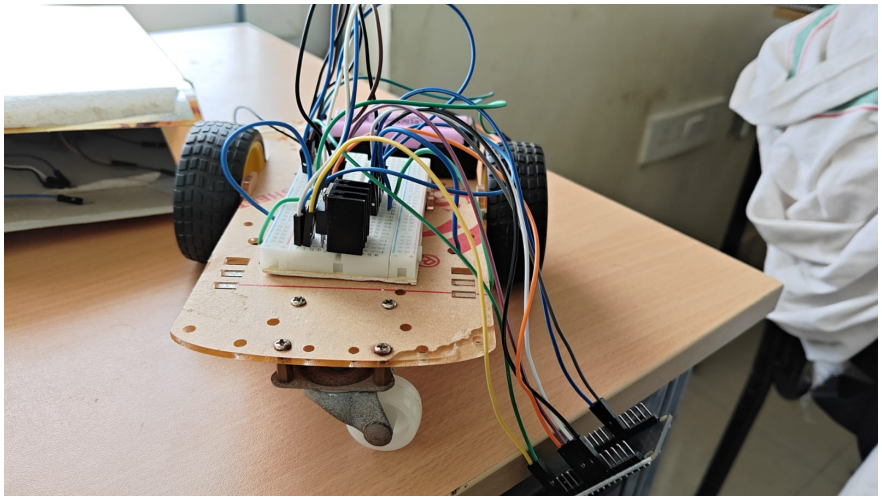


Figure 9: Updated wiring using the regulator

ESP32 board support. I installed the ESP32 board package from the Boards Manager in the Arduino IDE (Figure 10).



Figure 10: Installing the ESP32 board package in the Arduino IDE

Entry 5: Independent PWM speed control

The following Arduino program implements independent PWM speed control for the left and right motors using the ESP32 and L293D.

Source code: `https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/PWM_testing.ino`

Entry 6 : Problems encountered: right motor and power supply

Symptoms. (1) The right wheel did not rotate when the code ran. (2) The ESP32 did not power on from the battery through the regulator.

Diagnosis: right motor. I first checked the wires from each motor: the left wheel worked as intended and the right did not. Inspecting the right motor, I found that one soldered joint had broken off. After resoldering, it still did not work, because the wire to that joint had also broken and was already very short. While joining the short wires, the wire's joint with the motor came off again. After re-making the joints, both motors worked as intended.

Diagnosis: power supply. The ESP32 is meant to run without USB, but here the regulator became hot, the motors worked, and the ESP32 stayed off. When I connected the ESP32 VIN pin directly to the regulator's input pin, it turned on. On checking the part, I found that I had an **IC 7905 instead of a 7805**.

Root cause. The 7905 is a *negative* 5 V regulator with a different pinout, so it could not supply +5 V to the ESP32.

Fix. I have to get a 7805 from core labs.

Lesson. Check part markings and measure regulator output with a multimeter before connecting the ESP32.

Entry 7: Results of independent PWM speed control

Both wheels can operate at different PWM values, which gives fine control over the motors. I also verified that the movement commands work as listed in Table 4.

Table 4: Basic movement test

Movement	Left motor	Right motor	Observed result
Forward	Forward	Forward	UGV moved forward.
Backward	Backward	Backward	UGV moved backward.
Left pivot	Reverse	Forward	UGV pivoted to the left.
Right pivot	Forward	Reverse	UGV pivoted to the right.
Stop	Stop	Stop	Both motors stopped and the UGV remained stationary.

Entry 8: Bluetooth connection with the Android phone

This code verifies communication between the phone and the ESP32.

Source code: `https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/bluetooth_testing.ino`

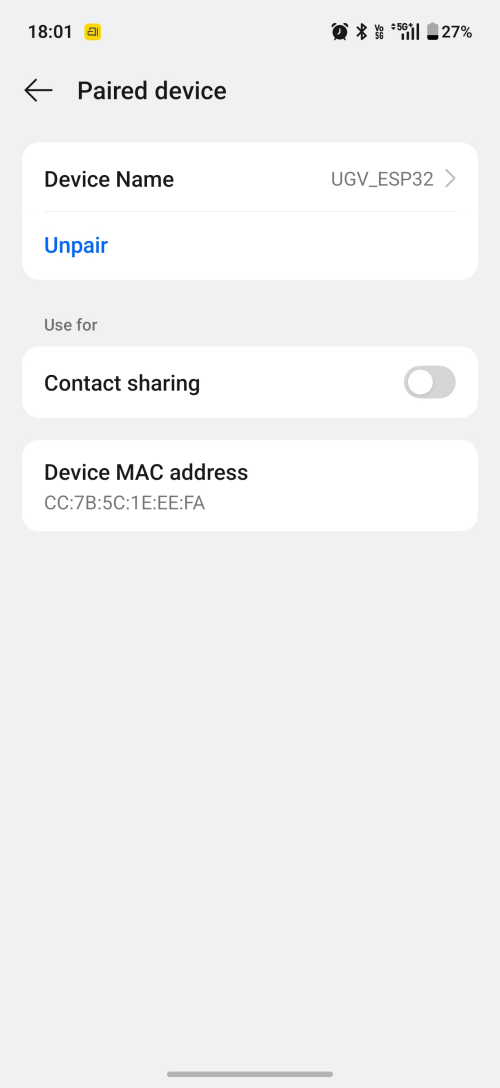


Figure 11: Pairing with the ESP32 over Bluetooth

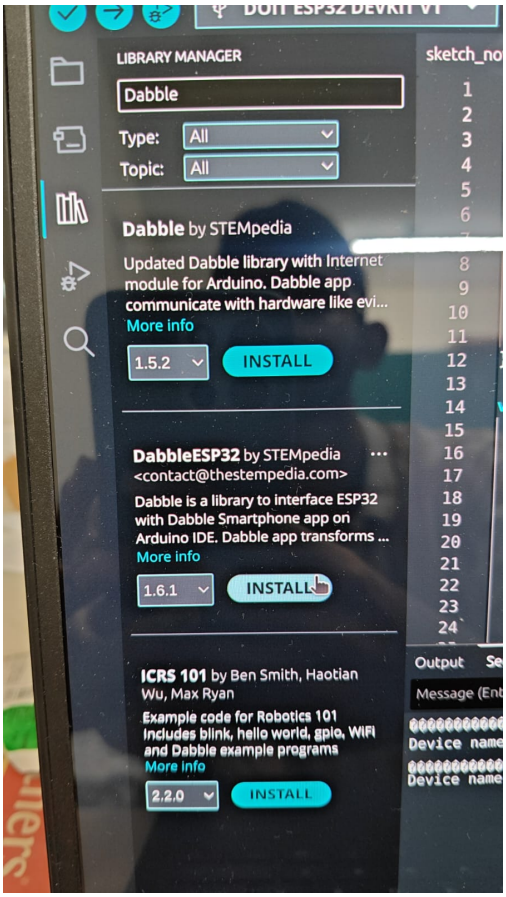


Figure 12: Downloading the Dabble library in the Arduino IDE

The ESP32 connected to the Dabble app over Bluetooth, and I checked that every button press was read correctly (Figure 13). The controls were tested with the following code:

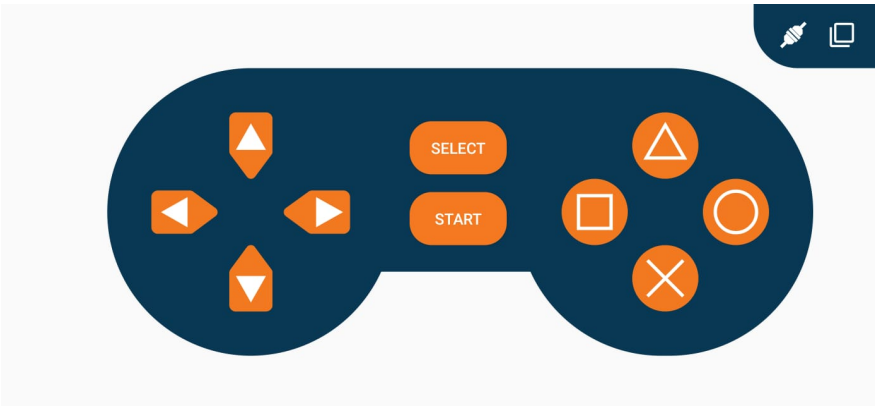


Figure 13: Gamepad module in the Dabble app

Source code: https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/controls_testing.ino

Entry 9: Moving the UGV with the Dabble gamepad

Code used for this step:

Source code: https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/moving_using_controls.ino

The code supported three speeds:

Table 5: Initial speed levels

Speed	PWM (0–255)
Slow	100
Medium	160
Fast	220

Observation. The UGV only moved at the Fast setting; below about 160 PWM the motors did not start. Finding the exact minimum starting PWM for each motor is needed before meaningful slow and medium speeds can be defined.

Entry 10: Change of plan

Calibrating exact PWM values for slow, medium and fast for each individual motor would take too long. I therefore **replaced the three fixed speeds with a user-adjustable PWM**, controlled from the gamepad as in Table 6. This also handles the minimum-starting-PWM problem, since the user can tune the PWM live.

Table 6: Bluetooth button controls for PWM adjustment

Dabble button	Action	PWM change
Square	Increase PWM by 10	+10
Triangle	Increase PWM by 1	+1
Circle	Decrease PWM by 10	−10
X	Decrease PWM by 1	−1

Entry 11: Smooth acceleration and deceleration

Abrupt changes in PWM cause sudden current spikes and wheel slip, and put stress on the motors and gears. I therefore made the PWM change gradually over 1.5 s. The code keeps four variables, `currentPWMLeft`, `targetPWMLeft`, `currentPWMRight` and `targetPWMRight`, and moves each `current` value linearly towards its `target` using `micros()`. Because it uses `micros()` rather than `delay()`, the loop is non-blocking and Bluetooth inputs are still read during a ramp.

Source code: https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/smooth_acc.ino

Entry 12: Mechanical damage and repair

My UGV fell to the ground during testing and the battery cable was cut in half. I patched it by soldering, after which nothing worked.

Diagnosis. I tested the motors and they did not work. I took everything apart. The motor solder joints were intact, and the ESP32 was functioning properly (visible on the serial monitor). Powering the motors from the ESP32 VIN and GND pins showed that the motors themselves were fine. This isolated the fault to the battery cable I had patched.

Root cause and fix. The patched cable was not conducting because of a faulty repair. I removed it and re-made the connection, and the UGV then worked as intended.

Lessons. Secure the battery pack in a holder, add strain relief to the cable, and insulate soldered joints (e.g. with heat shrink). Test continuity with a multimeter after any repair.

Entry 13: Bluetooth reconnection and fail-safe

I improved the code so that:

- the ESP32 is available to connect over Bluetooth every time it is powered on;
- if the Bluetooth connection drops, the UGV performs an emergency stop (fail-safe);
- the emergency stop can also be toggled manually with the Start button, and reset with the Select button.

Source code: `https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/bluetooth_reconnect.ino`

3 Current Status

Completed in Part 1.

- Two-wheel differential-drive UGV wired and working.
- Independent PWM control of both motors; forward, reverse, left, right and stop.
- Bluetooth control from an Android phone via Dabble.
- User-adjustable PWM instead of fixed speed levels.
- Smooth acceleration and deceleration.
- Fail-safe emergency stop on Bluetooth loss.

Deviations from the plan. Fixed speed levels were replaced with adjustable PWM (Entry 2)[motor calibration revealed that the minimum PWM for rotation of both wheels are nearly the same]

The final uploadable code is -

Source code: `https://github.com/yashrajkranthi22-jpg/EE1010-Project-Submissions-2026/blob/main/ee26btech11235_ee26btech11224/UGV/codes/bluetooth_reconnect.ino`